Experiment 1: 8-Puzzle

Aim:

Solve the 8-puzzle problem.

Algorithm:

1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
from queue import Queue


goal = [[1,2,3],[4,5,6],[7,8,0]]


start = [[1,2,3],[4,0,6],[7,5,8]]


def find_zero(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j


def copy(state):
    return [row[:] for row in state]


def bfs(start):
    q = Queue()
    q.put(start)
    visited = []
```

```
    while not q.empty():
        state = q.get()
        print(state)

        if state == goal:
            print("Goal Reached")
            return

        visited.append(state)

        x, y = find_zero(state)

        moves = [(1,0),(-1,0),(0,1),(0,-1)]

        for dx, dy in moves:
            nx, ny = x+dx, y+dy
            if 0 <= nx < 3 and 0 <= ny < 3:
                new = copy(state)
                new[x][y], new[nx][ny] = new[nx][ny], new[x][y]
                if new not in visited:
                    q.put(new)

bfs(start)
```

Output

```
============================== RESTART: E
Goal Reached
((1, 2, 3), (4, 0, 6), (7, 5, 8))
((1, 2, 3), (4, 5, 6), (7, 0, 8))
((1, 2, 3), (4, 5, 6), (7, 8, 0))
```

Result:
Goal state reached.

Experiment 2: 8-Queen

Aim:

Place 8 queens safely.

Algorithm:

1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
N = 8


board = [[0]*N for _ in range(N)]


def safe(row, col):
    for i in range(col):
        if board[row][i]:
            return False


    i, j = row, col
    while i >= 0 and j >= 0:
        if board[i][j]:
            return False
        i -= 1
        j -= 1


    i, j = row, col
    while i < N and j >= 0:
        if board[i][j]:
```

```
            return False
        i += 1
        j -= 1

    return True

def solve(col):
    if col >= N:
        return True

    for i in range(N):
        if safe(i, col):
            board[i][col] = 1
            if solve(col+1):
                return True
            board[i][col] = 0

    return False

solve(0)

for row in board:
    print(row)
```

Output

```
[1, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 1, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 1]
[0, 1, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 1, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 1, 0, 0]
[0, 0, 1, 0, 0, 0, 0, 0]
```

Result:
Valid solution displayed.

Experiment 3: Water Jug

Aim:

Obtain exactly 2 litres.

Algorithm:

1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
from collections import deque


visited = set()


queue = deque()


queue.append((0,0))


while queue:
    x, y = queue.popleft()


    if (x,y) in visited:
        continue


    visited.add((x,y))


    print(x, y)


    if x == 2:
        print("Goal Reached")
```

```
        break


    states = [
        (4,y),
        (x,3),
        (0,y),
        (x,0),
        (x-min(x,3-y), y+min(x,3-y)),
        (x+min(y,4-x), y-min(y,4-x))
    ]


    for s in states:
        if s not in visited:
            queue.append(s)
```

Output

```
(0, 0)
(4, 0)
(0, 3)
(4, 3)
(1, 3)
(3, 0)
(1, 0)
(3, 3)
(0, 1)
(4, 2)
(4, 1)
(0, 2)
(2, 3)
Goal Reached
```

Result:
Goal reached.

Experiment 4: Crypt-Arithmetic

Aim:

Solve SEND+MORE=MONEY.

Algorithm:

1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
from itertools import permutations


letters = "SENDMORY"


digits = range(10)


for p in permutations(digits,8):
    d = dict(zip(letters,p))


    if d['S']==0 or d['M']==0:
        continue


    send = d['S']*1000+d['E']*100+d['N']*10+d['D']
    more = d['M']*1000+d['O']*100+d['R']*10+d['E']
    money = d['M']*10000+d['O']*1000+d['N']*100+d['E']*10+d['Y']


    if send + more == money:
        print(d)
        print(send, "+", more, "=", money)
        break
```

Output

```
---------- RESTART: E:/AI Exp 4.py ----------
Solution Found
SEND = 9567
MORE = 1085
MONEY = 10652
Letter Mapping:
{'S': 9, 'E': 5, 'N': 6, 'D': 7, 'M': 1, 'O': 0, 'R': 8, 'Y': 2}
```

Result:

Correct assignment displayed.

Experiment 5: Missionaries and Cannibals

Aim:

Move everyone safely.

Algorithm:

1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
from collections import deque


start = (3,3,1)

goal = (0,0,0)


queue = deque([start])


visited = set()


moves = [(1,0),(2,0),(0,1),(0,2),(1,1)]


while queue:


    m,c,b = queue.popleft()


    if (m,c,b) in visited:

        continue


    visited.add((m,c,b))
```

```
    print((m,c,b))

    if (m,c,b)==goal:
        print("Goal Reached")
        break

    for dm,dc in moves:

        if b==1:
            nm,nc,nb = m-dm,c-dc,0
        else:
            nm,nc,nb = m+dm,c+dc,1

        if 0<=nm<=3 and 0<=nc<=3:
            if (nm==0 or nm>=nc) and ((3-nm)==0 or (3-nm)>=(3-nc)):
                queue.append((nm,nc,nb))
```

Output

```
(3, 3, 1)
(3, 2, 0)
(3, 1, 0)
(2, 2, 0)
(3, 2, 1)
(3, 0, 0)
(3, 1, 1)
(1, 1, 0)
(2, 2, 1)
(0, 2, 0)
(0, 3, 1)
(0, 1, 0)
(1, 1, 1)
(0, 2, 1)
(0, 0, 0)
Goal Reached
```

Result:
Goal reached.

Experiment 6: Vacuum Cleaner

Aim:

Clean all rooms.

Algorithm:

1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
rooms = ['Dirty','Dirty']


position = 0


while 'Dirty' in rooms:


    if rooms[position]=='Dirty':

        print("Cleaning Room",position)

        rooms[position]='Clean'


    position = 1-position


print("All Rooms Clean")

print(rooms)
```

Output

```
Current Room: 0
Cleaning Room 0
Current Room: 1
Cleaning Room 1

All Rooms are Clean
Final State: ['Clean', 'Clean']
```

Result:
All rooms cleaned.

Experiment 7: Breadth First Search

Aim:

Traverse a graph using BFS.

Algorithm:

1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
graph = {

'A':['B','C'],

'B':['D','E'],

'C':['F'],

'D':[],

'E':['F'],

'F':[]

}


visited = []


queue = ['A']


while queue:


    node = queue.pop(0)


    if node not in visited:


        print(node,end=" ")
```

```
        visited.append(node)


        queue.extend(graph[node])
```

Output

```
BFS Traversal:
A B C D E F
```

Result:
Traversal displayed.

Experiment 8: Depth First Search

Aim:

Traverse a graph using DFS.

Algorithm:

1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
graph = {

'A':['B','C'],

'B':['D','E'],

'C':['F'],

'D':[],

'E':['F'],

'F':[]

}


visited = set()


def dfs(node):


   if node not in visited:


       print(node,end=" ")


       visited.add(node)


       for i in graph[node]:
```

dfs(i)

dfs('A')

Output

```
DFS Traversal:
A B D E F C
```

Result:
Traversal displayed.

Experiment 9: Travelling Salesman Problem

Aim:

Find the minimum tour cost.

Algorithm:

1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
from itertools import permutations


graph = [

[0,10,15,20],

[10,0,35,25],

[15,35,0,30],

[20,25,30,0]

]


cities = [1,2,3]


min_cost = 999


for p in permutations(cities):


    cost = 0


    k = 0


    for j in p:
```

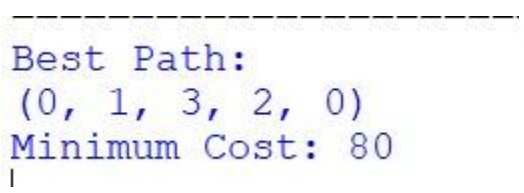
```
        cost += graph[k][j]
        k = j

    cost += graph[k][0]

    min_cost = min(min_cost,cost)

print("Minimum Cost =",min_cost)
```

Output



Result:
Minimum cost displayed.

Experiment 10: A* Algorithm

Aim:

Find the shortest path.

Algorithm:

1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
graph = {

'A':[('B',1),('C',3)],

'B':[('D',3),('E',1)],

'C':[('F',5)],

'D':[],

'E':[('F',1)],

'F':[]

}


heuristic = {

'A':6,

'B':4,

'C':2,

'D':3,

'E':1,

'F':0

}


open_list = [('A',0)]
```

```
visited = []

while open_list:

    open_list.sort(key=lambda x:x[1]+heuristic[x[0]])

    node,cost = open_list.pop(0)

    if node in visited:
        continue

    print(node)

    visited.append(node)

    if node=='F':
        print("Goal Reached")
        break

    for n,w in graph[node]:
        open_list.append((n,cost+w))
```

Output

```
Visited: A
Visited: B
Visited: E
Visited: F
Goal Reached
Total Cost: 3
```

Result:
Goal reached.

Experiment 11: Map Coloring (CSP)

Aim:

Colour a map without conflicts.

Algorithm:

1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
colors=['Red','Green','Blue']
graph={'A':['B','C'],'B':['A','C'],'C':['A','B']}
res={}
def safe(n,c): return all(res.get(x)!=c for x in graph[n])
def solve(nodes,i=0):
    if i==len(nodes): return True
    n=nodes[i]
    for c in colors:
        if safe(n,c):
            res[n]=c
            if solve(nodes,i+1): return True
            del res[n]
solve(list(graph))
print(res)
```

Result:

Valid colouring displayed.

Experiment 12: Tic Tac Toe

Aim:

Implement Tic Tac Toe game.

Algorithm:

1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
board=[' ']*9
def show():
   print(board[0:3]);print(board[3:6]);print(board[6:9])
player='X'
while ' ' in board:
   show()
   p=int(input('Position: '))-1
   if board[p]==' ':
      board[p]=player
      player='O' if player=='X' else 'X'
```

Result:

Game executed.